In [ ]:
```
import os
import re
from glob import glob

import pandas as pd
from openai import OpenAI
import tiktoken
```

In [ ]:
```
definitions = {
    "Fiscal Policy": "the use of government spending, taxation, and borrowing to influence a country's eco
    "Debt and Deficits": "two key aspects of government finance. Debt is the total amount of money a gover
    "Monetary Policy": "the actions undertaken by a nation's central bank or monetary authority to manage 
    "Inflation": "the rate at which the general level of prices for goods and services rises over time, le
    "Supply Shocks": "an unexpected event that suddenly changes the availability of goods or services, sig
    "Demand Shocks": "a sudden and unexpected change in the level of demand for goods and services within 
    "Climate Change": "significant, long-term changes in the average weather patterns on Earth, primarily 
    "Geopolitics": "the study of how geographic space—including location, resources, and physical terrain—
    "Housing Market": "refers to the supply and demand for residential properties, encompassing the buying
    "Financial Stability": "a condition in which the financial system—including institutions, markets, and
    "Stock Market": "a financial marketplace where investors buy and sell shares of publicly traded compan
}
```

In [ ]:

In [ ]:
```
# root = "C:/Users/lihou/Box/PoliticsCentralBanking"
# root = '/Users/fred/Library/CloudStorage/Box-Box/PoliticsCentralBanking'
```

In [ ]:
```
# # include subfolders as well
# # csvs = glob(f'{root}/data/ProQuest/Proofread_files/**/*.csv', recursive=True)

# # not include subfolder
# csvs = glob(f'{root}/data/ProQuest/Proofread_files/*.csv', recursive=True)

# # sort csvs
# csvs = sorted(csvs)

# df = pd.DataFrame()
# for csv in csvs:
#     try:
```

```
#         df_temp = pd.read_csv(csv, encoding='utf-8')
#         df_temp['file'] = os.path.basename(csv)
#         df = pd.concat([df, df_temp])
#     except UnicodeDecodeError:
#         df_temp = pd.read_csv(csv, encoding='latin1')
#         df_temp['file'] = os.path.basename(csv)
#         df = pd.concat([df, df_temp])

# len(csvs)
```

In [10]:
```
# df = df[[x for x in df.columns if 'Unnamed' not in x and "RECHECK" not in x]]
```

In [11]:
```
# len(df)
```

In [12]:
```
# df['year'] = df['file'].str.extract(r'(\d{4})')
```

In [13]:
```
# for label in definitions.keys():
#     df[f'{label}_ans'] = ""
```

In [14]:
```
# df.to_csv(fr"{root}/estimation/topic-models/2024-10-25/df_step1.csv", index=False)
```

In [15]:
```
# df = pd.read_csv(fr"{root}/estimation/topic-models/2024-10-25/df_step1.csv")
```

In [16]:
```
# df = pd.read_csv(fr"{root}/estimation/topic-models/2024-10-25/df_step2_temp.csv")
```

In [17]:
```
df = pd.read_csv(rf"{root}/estimation/topic-models/2024-10-25/df_step2.csv")
```

```
/var/folders/rf/jrsrkjxn4rl0m0bmrmpys7b40000gq/T/ipykernel_34660/328716424.py:1: DtypeWarning: Columns (6)
have mixed types. Specify dtype option on import or set low_memory=False.
  df = pd.read_csv(rf"{root}/estimation/topic-models/2024-10-25/df_step2.csv")
```

In [18]:
```
# for label in definitions.keys():
#     df[f'{label}_ans'] = df[f'{label}_ans'].fillna("")
```

In [ ]:
```
# for label in definitions.keys():
#     df.loc[17603, f'{label}_ans'] = ''
```

In [19]:
```
def chatgpt(api_key, model, temp, prompt, system_message=None):
    client = OpenAI(api_key=api_key)
```

```
    model = model
    prompt = prompt

    messages = []
    if system_message:
        messages.append({"role": "system", "content": system_message})
    messages.append({"role": "user", "content": prompt})

    chat_completion = client.chat.completions.create(
        messages=messages, model=model, temperature=temp
    )
    answer = chat_completion.choices[0].message.content

    return answer
```

In [ ]:

```
# import time

# model = "gpt-4o-mini"
# # encoding = tiktoken.encoding_for_model(model)

# max_retry_count = 10

# # get the row number where left over
# row_unprocessed = df[df['Fiscal Policy_ans'] == ""].index[0]
# print(f"Starting from row {row_unprocessed}")
# last_save_time = time.time()

# for i, row in df[row_unprocessed:].iterrows():
#     text = row['text']
#     # input_token_count = 0
#     # output_token_count = 0

#     for label in definitions.keys():
#         definition = definitions[label]

#         system_message = f"You are a helpful assistant helping me to verify if a speech I am about to pr

#         query = text if pd.notna(text) else ""
#         retry_count = 0
#         success = False

#         while retry_count < max_retry_count and not success:
```

```
#             try:
#                 answer = chatgpt(jingoo_api_key, model, 0, query, system_message)
#                 # input_token_count += len(encoding.encode(query))
#                 # output_token_count += len(encoding.encode(answer))
#                 df.at[i, f'{label}_ans'] = answer
#                 success = True
#             except Exception as e:
#                 retry_count += 1
#                 print(f"Error: {e}, retrying {retry_count}/{max_retry_count} for index {i}, label {label}
#                 time.sleep(5)

#                 if retry_count >= max_retry_count:
#                     print(f"Max retries reached for index {i}, label {label}. Skipping this item.")
#                     break

#     # df.at[i, 'query_token_count'] = input_token_count
#     # df.at[i, 'output_token_count'] = output_token_count

#     if i % 50 == 0:
#         current_time = time.time()
#         elapsed_time = current_time - last_save_time
#         last_save_time = current_time
#         df.to_csv(fr"{root}/estimation/topic-models/2024-10-25/df_step2_temp.csv", index=False)
#         print(f"Saved at index {i}. Time since last save: {elapsed_time:.2f} seconds")
```

In [21]:
```
# answer_list = []
# for label in definitions.keys():
#     answer_list += df[f'{label}_ans'].unique().tolist()
```

In [22]:
```
# answer_list = [x for x in answer_list if x != '' and pd.notna(x)]

# # drop duplicates
# answer_list = list(set(answer_list))
```

In [23]:
```
# answer_list = sorted(answer_list)
```

In [24]:
```
# with open(fr"{root}/estimation/topic-models/2024-10-25/answer_list.txt", "w") as f:
#     for item in answer_list:
#         f.write(f"{item}\n")
```

In [25]:
```
# answer_list
```

In [26]:
```
# with open(fr"{root}/estimation/topic-models/2024-10-25/valid_answer_list.txt", "r") as f:
#     valid_answer_list = f.readlines()

# valid_answer_list = [x.strip() for x in valid_answer_list]
```

In [27]:
```
# valid_answer_list[-5:]
```

In [28]:
```
# for label in definitions.keys():
#     df[f'{label}_ans'] = df[f'{label}_ans'].apply(lambda x: x if x in valid_answer_list else "invalid")
```

In [29]:
```
# def parse_valid_answer(answer):
#     if answer == "invalid":
#         return "IV", ""

#     elif re.match(r"^No\.", answer):
#         return "0", ""
#     elif re.match(r"^Unsure\.", answer):
#         return "U", ""
#     else:
#         try:
#             answer_sub = re.sub(r"^Yes\.", "", answer).strip()
#             answer_sub = re.sub(r'^"|"$', "", answer_sub).strip()
#             return "1", answer_sub.strip()
#         except:
#             return 1, "Unable"
```

In [ ]:
```
# for label in definitions.keys():
#     df[[f'{label}_yn', f'{label}_sentence']] = df[f'{label}_ans'].apply(lambda x: parse_valid_answer(x))
```

In [31]:
```
# extract every element from [[f'{label}_ans', f'{label}_yn', f'{label}_sentence'] for label in definition
# col_list = ['section', 'speaker', 'text', 'meta_data', 'check_typos', 'file', 'year'] + [item for sublis
# df = df[col_list]
```

In [32]:
```
# df.to_csv(fr"{root}/estimation/topic-models/2024-10-25/df_step3.csv", index=False)
```

In [33]:
```
df = pd.read_csv(rf"{root}/estimation/topic-models/2024-10-25/df_step3.csv")
```

```
/var/folders/rf/jrsrkjxn4rl0m0bmrmpys7b40000gq/T/ipykernel_34660/1273472983.py:1: DtypeWarning: Columns (6)
have mixed types. Specify dtype option on import or set low_memory=False.
  df = pd.read_csv(rf"{root}/estimation/topic-models/2024-10-25/df_step3.csv")
```

In [34]:
```
# dfy means any row with a yes in yn column
dfy = df[
    df[[f"{label}_yn" for label in definitions.keys()]].apply(
        lambda x: x.str.contains("1").any(), axis=1
    )
]
sample_row = dfy.sample(1).T
```

In [35]:
```
# dfn means df with number yn column, excluding iv and unsure answers
dfn = df[
    df[[f"{label}_yn" for label in definitions.keys()]].apply(
        lambda x: x.str.contains("1|0").any(), axis=1
    )
]
```

In [36]:
```
for label in definitions.keys():
    dfn[f"{label}_yn"] = dfn[f"{label}_yn"].apply(lambda x: 1 if x == "1" else 0)
```

```
/var/folders/rf/jrsrkjxn4rl0m0bmrmpys7b40000gq/T/ipykernel_34660/3464782386.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html
#returning-a-view-versus-a-copy
  dfn[f"{label}_yn"] = dfn[f"{label}_yn"].apply(lambda x: 1 if x == "1" else 0)
```

In [37]:
```
dfn["count"] = 1
```

```
/var/folders/rf/jrsrkjxn4rl0m0bmrmpys7b40000gq/T/ipykernel_34660/1170261034.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html
#returning-a-view-versus-a-copy
  dfn["count"] = 1
```

In [130…
```
dfn_year = dfn.groupby("year").sum()

for label in definitions.keys():
    dfn_year[f"{label}_pct"] = dfn_year[f"{label}_yn"] / dfn_year["count"] * 100

# get the percentage that are both monetary and fiscal policy
dfn_year["Fiscal and Monetary Policy"] = dfn_year["Fiscal Policy_yn"] & dfn_year["Monetary Policy_yn"]
dfn_year['fiscal_monetary_pct'] = dfn_year["Fiscal and Monetary Policy"] / dfn_year["count"] * 100

dfn_year = dfn_year[[f"{label}_pct" for label in definitions.keys()] + ["count", "fiscal_monetary_pct"]]

dfn_year.drop(columns=["count"], inplace=True)
```

In [131…
```
dfn_year = dfn_year[1:-1]
```

In [132…
```
dfn_year['year'] = dfn_year.index
```

In [133…
```
dfn_year.iloc[10]['year'] # .year
```

Out[133… 1984

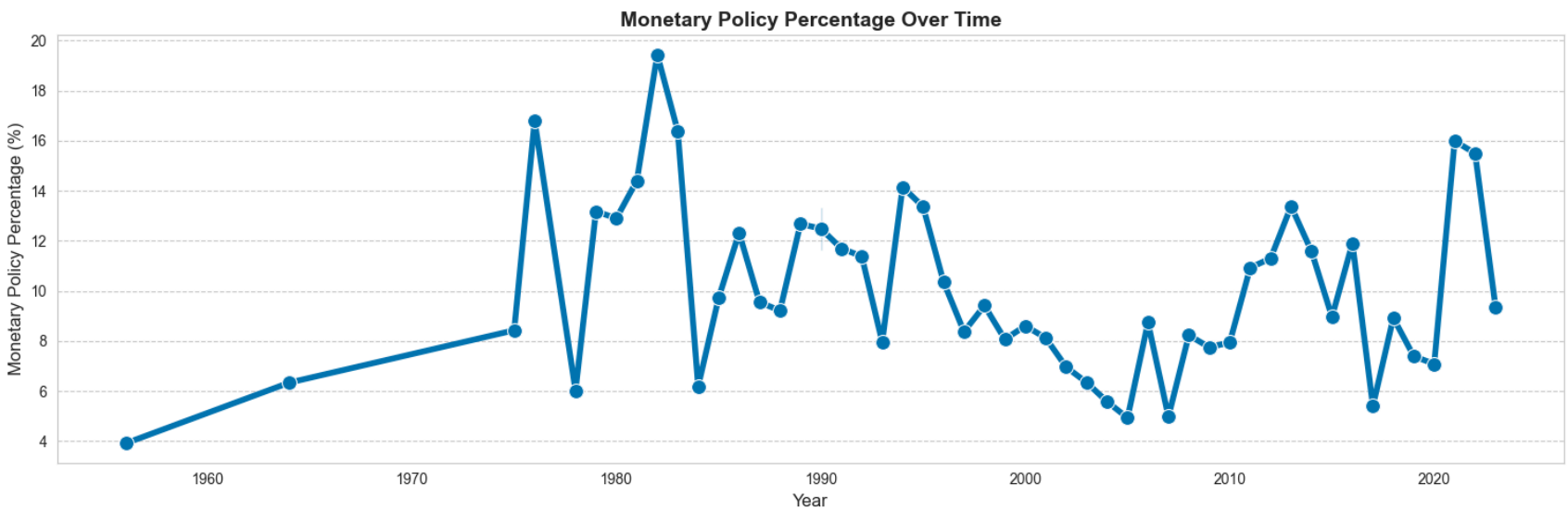
In [51]:
```
# dfn_year.to_csv(fr"{root}/estimation/topic-models/2024-10-25/dfn_year.csv")
```

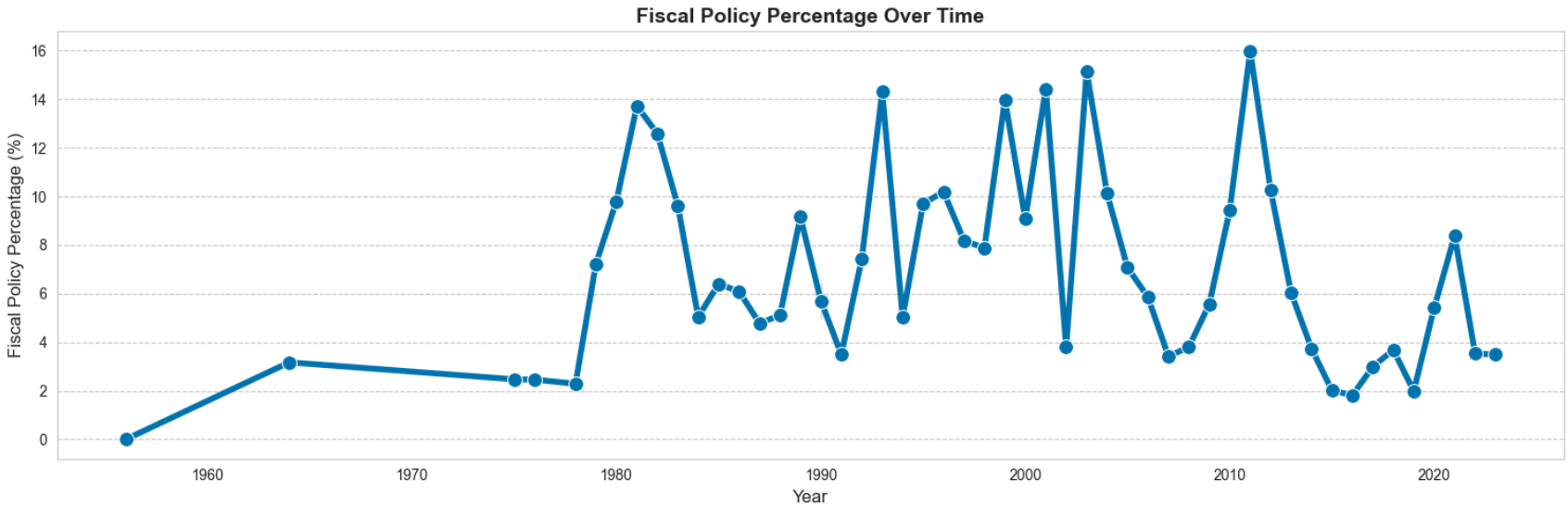
In [ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Set a clean style
sns.set_style("whitegrid")

# Convert index to datetime and sort it
dfn_year.index = pd.to_datetime(dfn_year.index, format="%Y")
dfn_year = dfn_year.sort_index()
time_series = dfn_year.index.year

num_labels = len(definitions.keys())
num_rows = (num_labels + 1) // 2  # Calculate the number of rows needed

# fig, axs = plt.subplots(num_rows, 2, figsize=(15, 5 * num_rows))
```

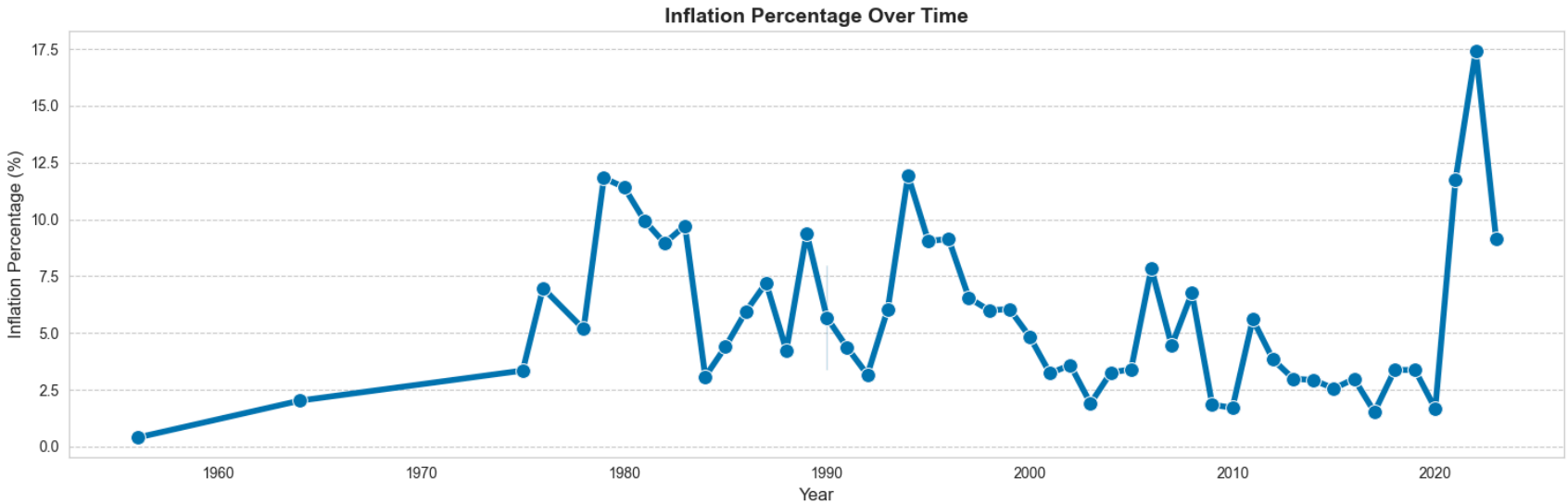
```
# # Add a title
# fig.suptitle(
#     "Frequency of Economic Terms in Speeches Over Time", fontsize=20, fontweight="bold"
# )

# range definition keys by how frequent they are
definitions_new = {
    k: v
    for k, v in sorted(
        definitions.items(),
        key=lambda item: dfn_year[f"{item[0]}_pct"].sum(),
        reverse=True,
    )
}

for i, label in enumerate(definitions_new.keys()):
    # set the figsize to be wide
    plt.figure(figsize=(15, 10))

    # Use a color palette for different plots
    sns.lineplot(
        x=time_series,
        y=dfn_year[f"{label}_pct"],
        # ax=axs[row, col],
        # color=sns.color_palette("tab10")[i % 10],
        marker='o',
        markersize=10,
        linewidth=4
    )

    fig = plt.gcf()
    ax = plt.gca()

    plt.title(
        f"{label} Percentage Over Time", fontsize=14, fontweight="bold"
    )
    plt.xlabel("Year", fontsize=12)
    plt.ylabel(f"{label} Percentage (%)", fontsize=12)

    # Add vertical dotted lines for years with a 500% increase within the next 2 years
    for j in range(len(dfn_year) - 2):  # Ensure we can look 2 years ahead
        all_time_high = dfn_year[f"{label}_pct"].max()
```

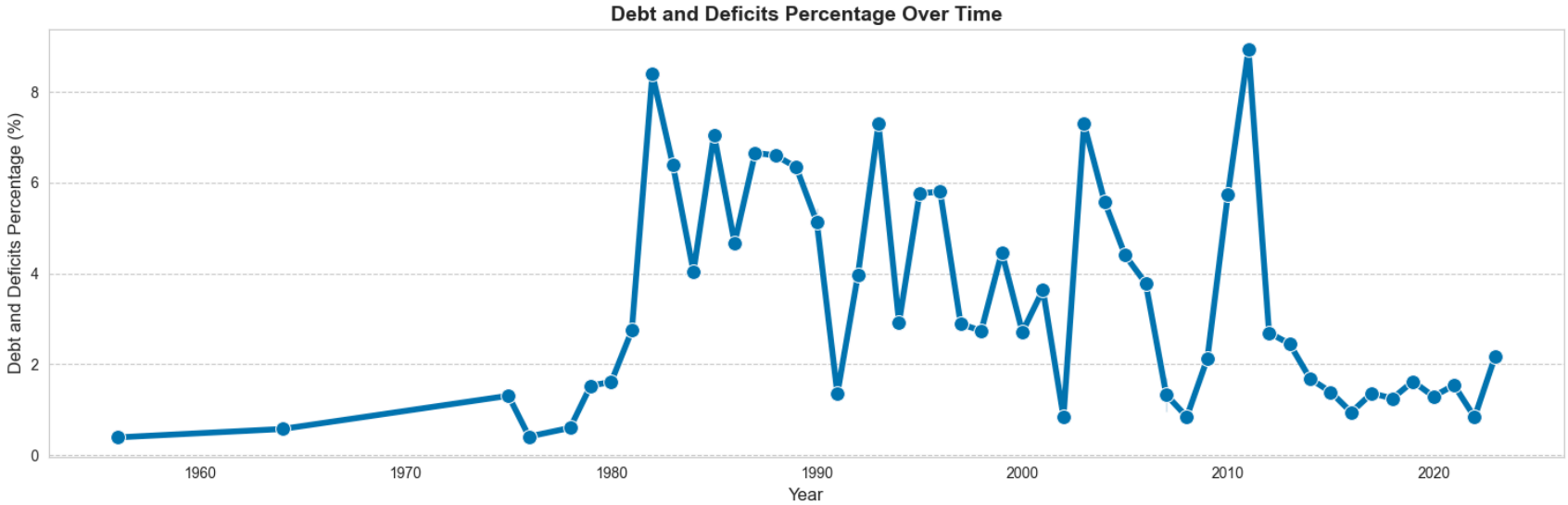
```
        current_value = dfn_year[f"{label}_pct"].iloc[j]
        future_value_1 = dfn_year[f"{label}_pct"].iloc[j + 1]
        future_value_2 = dfn_year[f"{label}_pct"].iloc[j + 2]
        year_j = dfn_year.iloc[j]['year']# .year

        cond1 = (
            future_value_2 >= 5 * current_value
            and future_value_1 >= 0.3 * all_time_high
        )
        cond2 = (
            future_value_1 >= 5 * current_value
            and future_value_2 >= 0.3 * all_time_high
        )
        cond3 = (
            future_value_2 >= 10 * current_value
            and current_value >= 0.1 * all_time_high
        )
        cond4 = (
            future_value_1 >= 10 * current_value
            and current_value >= 0.1 * all_time_high
        )

        # Special arrangements
        # cond5 = (i == 5) and ((year_j == 2005) or (year_j == 2003))
        # cond6 = (i == 7) and (year_j == 2007)
        # cond7 = (i == 6) and (year_j == 2001)
        # cond8 = (i == 3) and (year_j == 2008)
        # cond9 = (i == 2) and (year_j == 1992)

        # if cond1 or cond2 or cond3 or cond4:
        #     year_to_mark = dfn_year.index[j].year
        #     plt.axvline(
        #         year_to_mark, color="salmon", linestyle="--", linewidth=1
        #     )
        #     # Mark years at the top left of dotted lines with slight offset for readability
        #     plt.text(
        #         year_to_mark,
        #         # axs[row, col].get_ylim()[1] * 0.95,
        #         ax.get_ylim()[1] * 0.95,
        #         str(year_to_mark),
        #         verticalalignment="top",
        #         horizontalalignment="left",
```

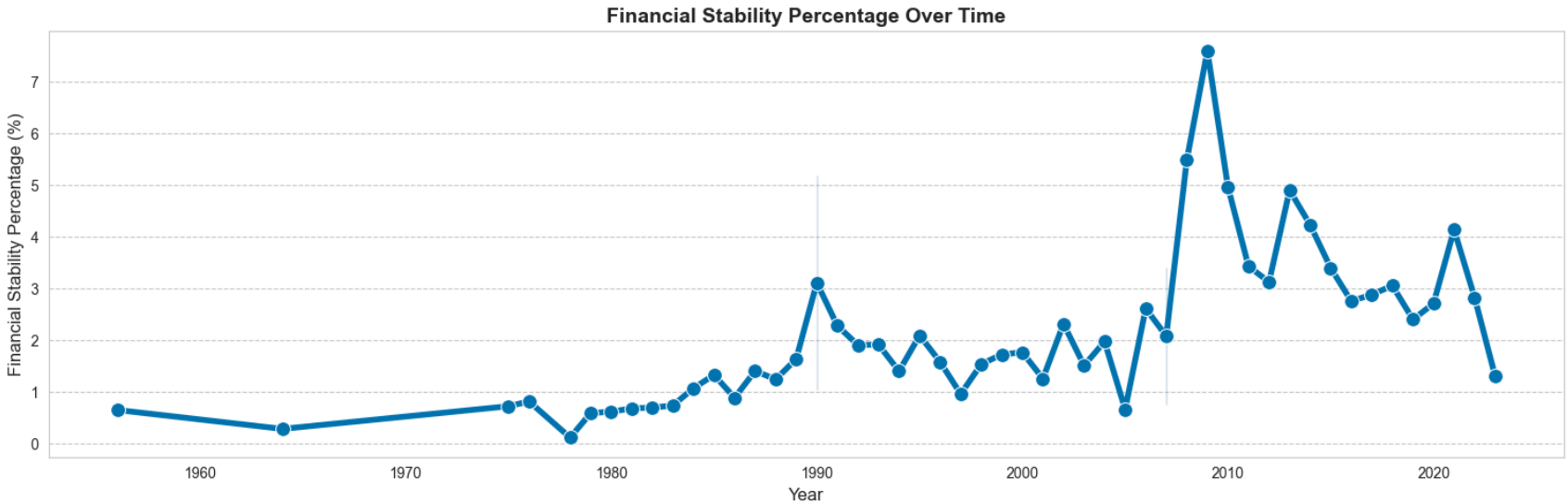
```
        #         color="black",
        #         fontsize=10,
        #         bbox=dict(
        #             facecolor="white",
        #             alpha=0.6,
        #             edgecolor="none",
        #             boxstyle="round,pad=0.3",
        #         ),
        #     )

        # clear the grid lines
        plt.grid(False)
        # use linestyle only y-axis
        plt.grid(axis='y', linestyle='--')

# # Remove any empty subplots
# for j in range(i + 1, num_rows * 2):
#     fig.delaxes(axs.flatten()[j])

    plt.tight_layout()

    # save the plot
    plt.savefig(fr"{root}/estimation/topic-models/2024-10-25/{label}_pct.png")

    plt.show()
```

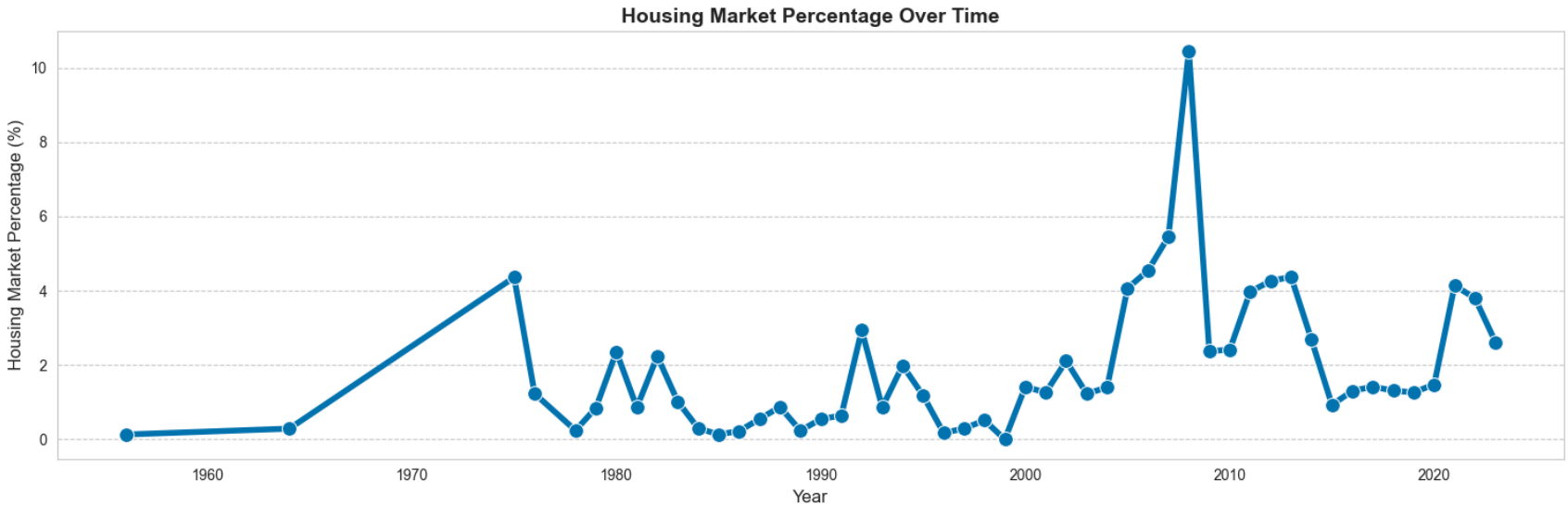


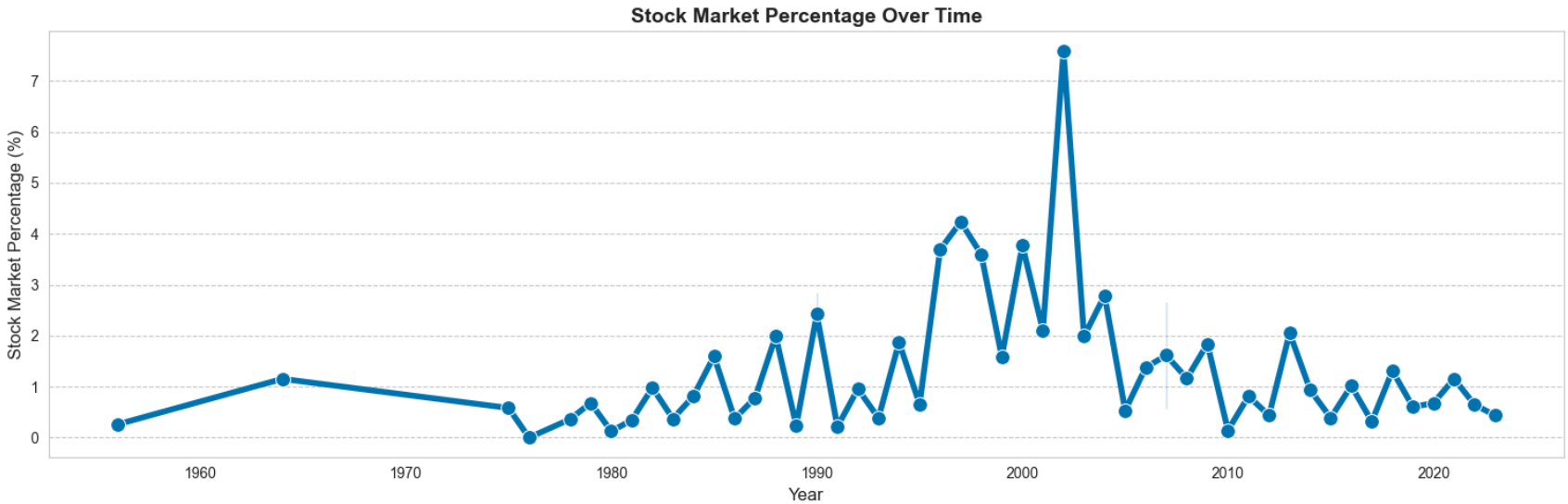

Fiscal Policy Percentage Over Time
Fiscal Policy Percentage (%)
0
2
4
6
8
10
12
14
16
1960
1970
1980
1990
2000
2010
2020
Year

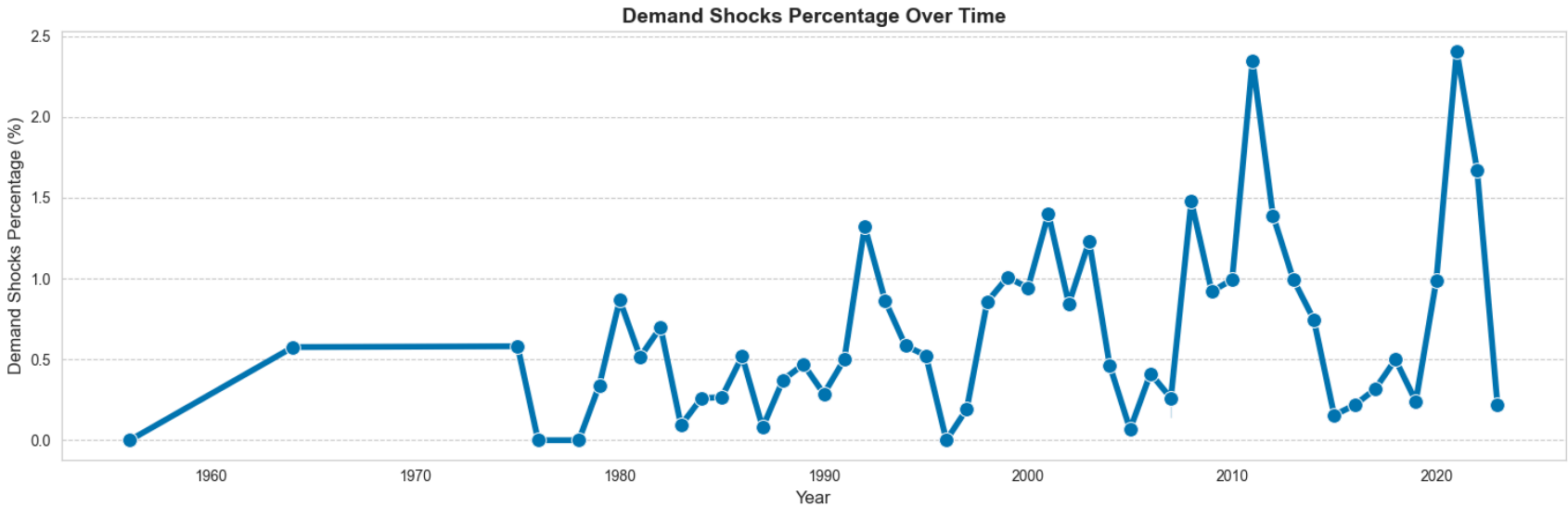

Inflation Percentage Over Time
Inflation Percentage (%)
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
1960
1970
1980
1990
2000
2010
2020
Year

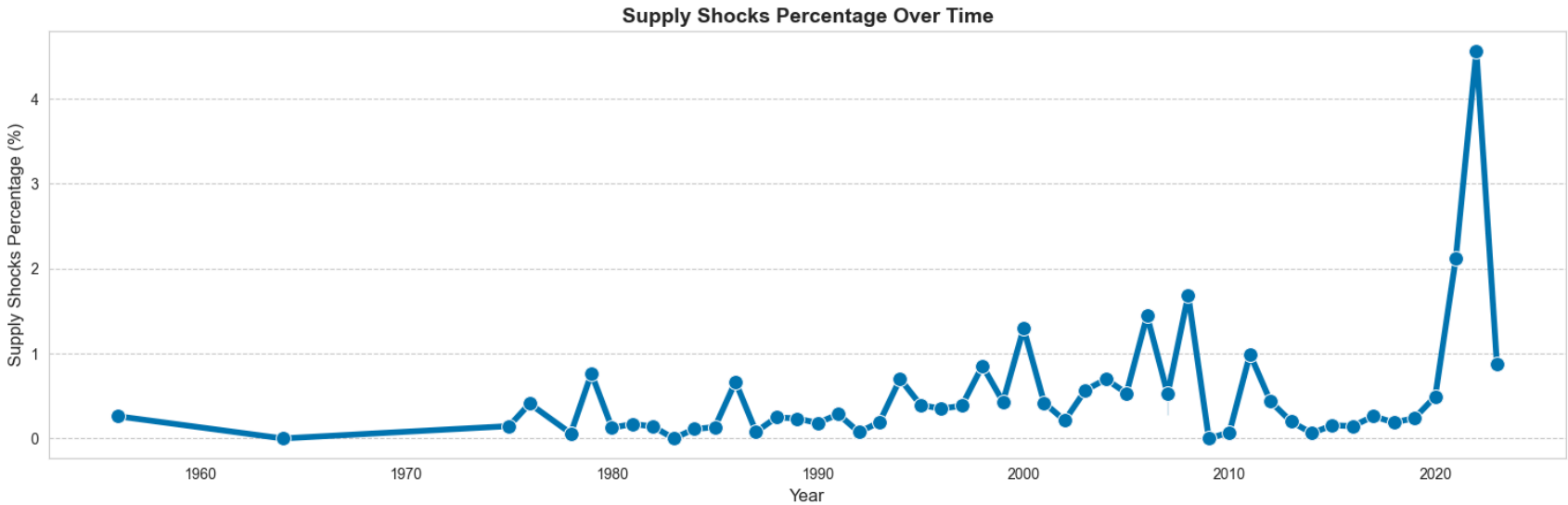

Debt and Deficits Percentage Over Time
Debt and Deficits Percentage (%)
0
2
4
6
8
1960
1970
1980
1990
2000
2010
2020
Year

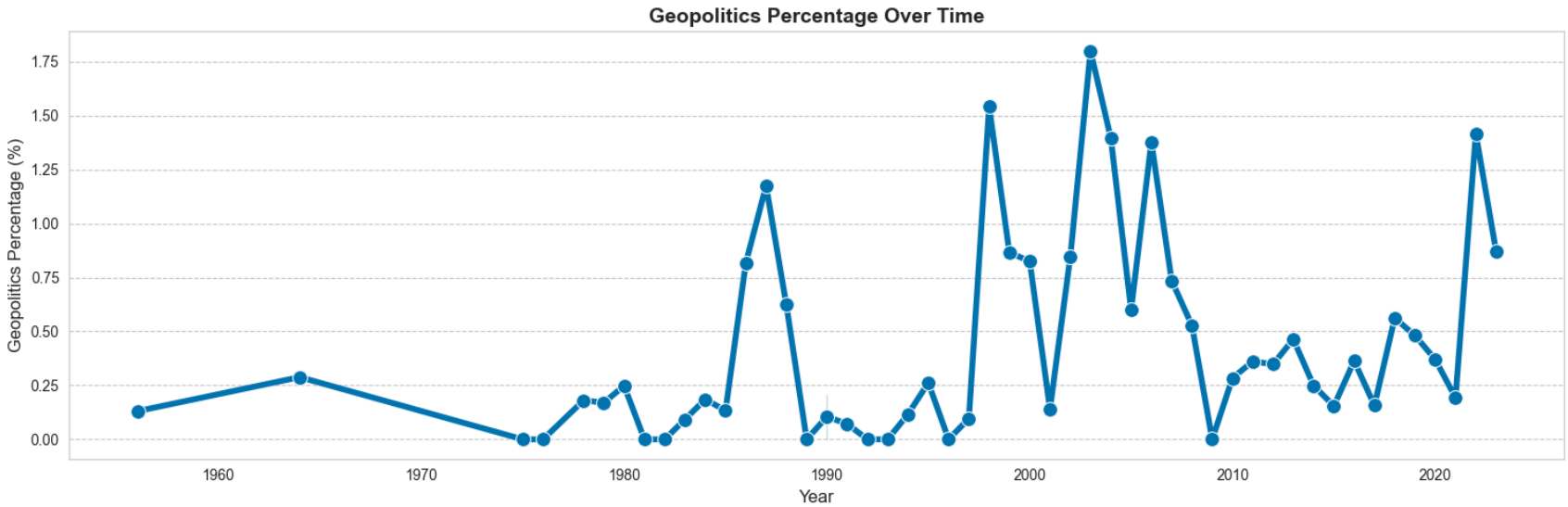

Financial Stability Percentage Over Time
Financial Stability Percentage (%)
0
1
2
3
4
5
6
7
1960
1970
1980
1990
2000
2010
2020
Year

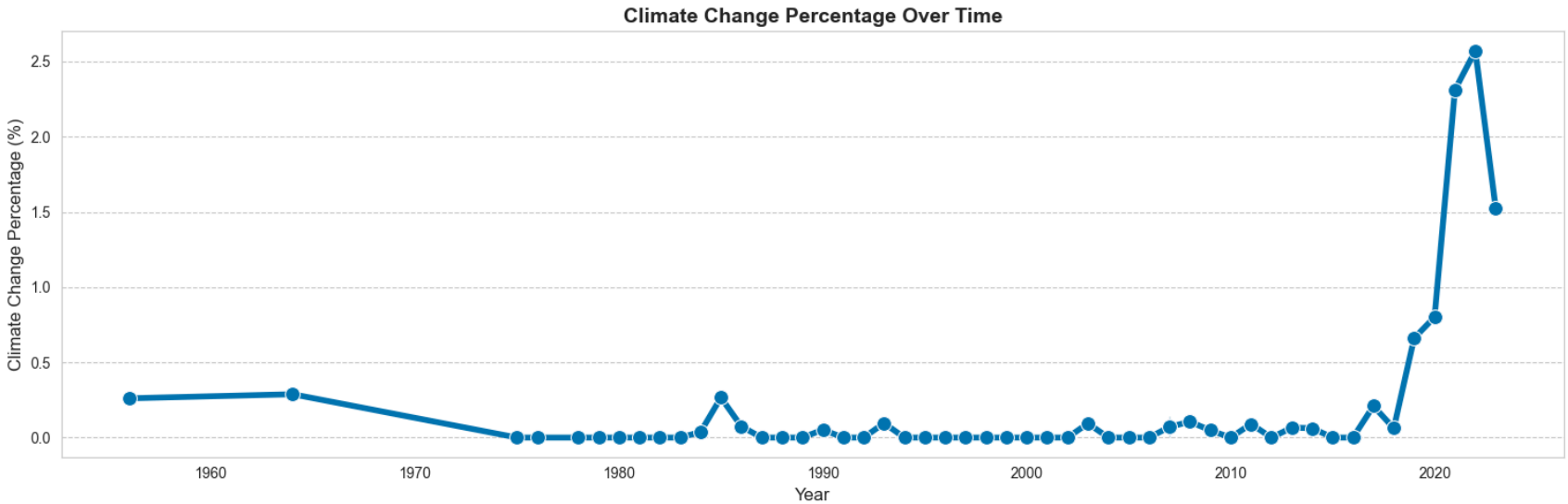

Housing Market Percentage Over Time
Housing Market Percentage (%)
0
2
4
6
8
10
1960
1970
1980
1990
2000
2010
2020
Year


Stock Market Percentage Over Time
Stock Market Percentage (%)
0
1
2
3
4
5
6
7
1960
1970
1980
1990
2000
2010
2020
Year


Demand Shocks Percentage Over Time
Demand Shocks Percentage (%)
2.5
2.0
1.5
1.0
0.5
0.0
1960
1970
1980
1990
2000
2010
2020
Year

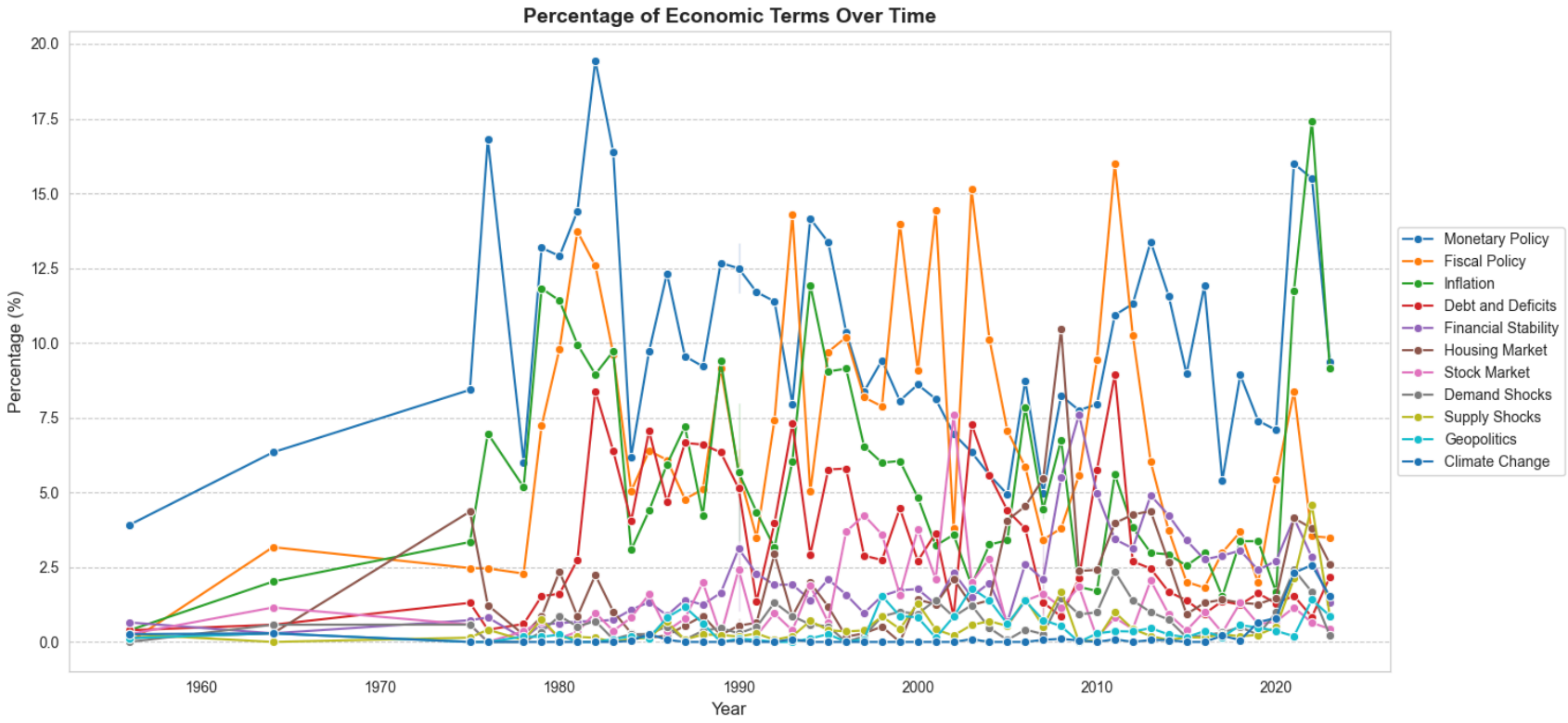

Supply Shocks Percentage Over Time
Supply Shocks Percentage (%)
4
3
2
1
0
1960
1970
1980
1990
2000
2010
2020
Year





In [80]:
```
# plot above, but all in one plot
plt.figure(figsize=(15, 7))

for i, label in enumerate(definitions_new.keys()):
    # Use a color palette for different plots
    sns.lineplot(
        x=time_series,
        y=dfn_year[f"{label}_pct"],
```

```
        # ax=axs[row, col],
        # color=sns.color_palette("tab10")[i % 10],
        marker='o',
        label=label
    )

    fig = plt.gcf()
    ax = plt.gca()

    plt.title(
        f"Percentage of Economic Terms Over Time", fontsize=14, fontweight="bold"
    )
    plt.xlabel("Year", fontsize=12)
    plt.ylabel("Percentage (%)", fontsize=12)

    # Add vertical dotted lines for years with a 500% increase within the next 2 years

    # clear the grid lines
    plt.grid(False)
    # use linestyle only y-axis
    plt.grid(axis='y', linestyle='--')

# set the legend to right side outside of the plot
plt.legend(loc='center left', bbox_to_anchor=(1, 0.5))
plt.tight_layout()
```

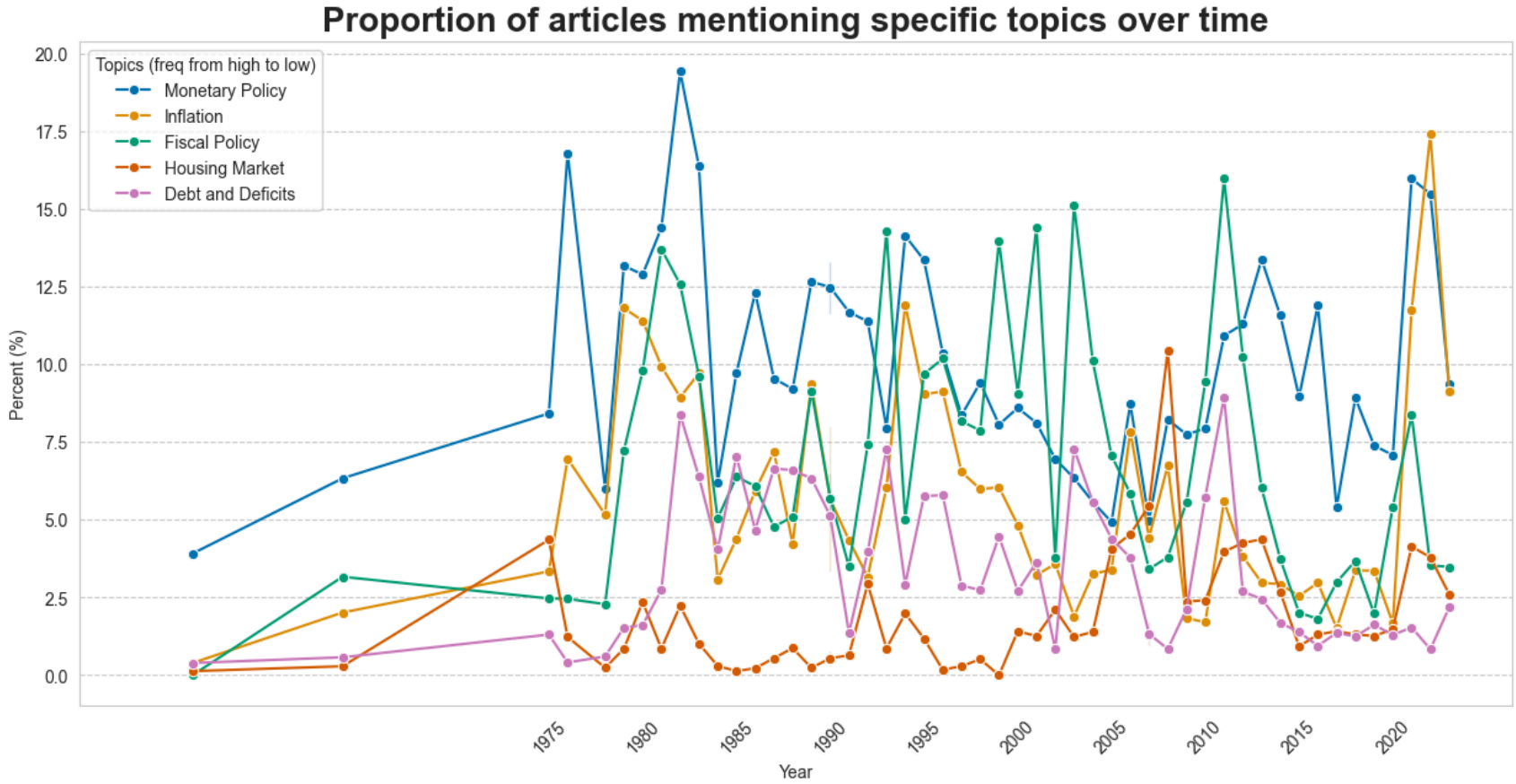


In [103…

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_palette("colorblind")
# plt.rcParams['font.family'] = 'Georgia'

dfn_year.index = pd.to_datetime(dfn_year.index, format="%Y")
dfn_year = dfn_year.sort_index()
time_series = dfn_year.index.year

last_year = 2023
last_year_data = dfn_year.loc[str(last_year)]

sorted_columns = last_year_data.mean().sort_values(ascending=False).index

plt.figure(figsize=(15, 7))

# drop "year" from sorted_columns
```

```python
sorted_columns = sorted_columns.drop("year")

for col in sorted_columns[:5]:
    # plt.plot(time_series, dfn_year[col], label=col)
    print(col)
    sns.lineplot(
        x=time_series,
        y=dfn_year[col],
        # ax=axs[row, col],
        # color=sns.color_palette("tab10")[i % 10],
        marker='o',
        label=f'{col.replace("_pct", "")}'
    )

plt.xlabel("Year")
plt.ylabel("Percent (%)")
plt.title(
    "Proportion of articles mentioning specific topics over time",
    fontdict={"fontsize": 20, "fontweight": "bold"},
)

plt.xticks(time_series, time_series.astype(int), rotation=45)

plt.legend(title="Topics (freq from high to low)", loc="upper left") # , bbox_to_anchor=(1, 0.7))

# increase the font size of the legend and the title
plt.setp(ax.get_legend().get_title(), fontsize='14')

# remove the grid
plt.grid(False)
# only show y-axis grid
plt.grid(axis='y', linestyle='--')

# Set major x-axis to be every year between 1950 and 2025
plt.xticks(range(1950, 2025, 5))


plt.show()
```

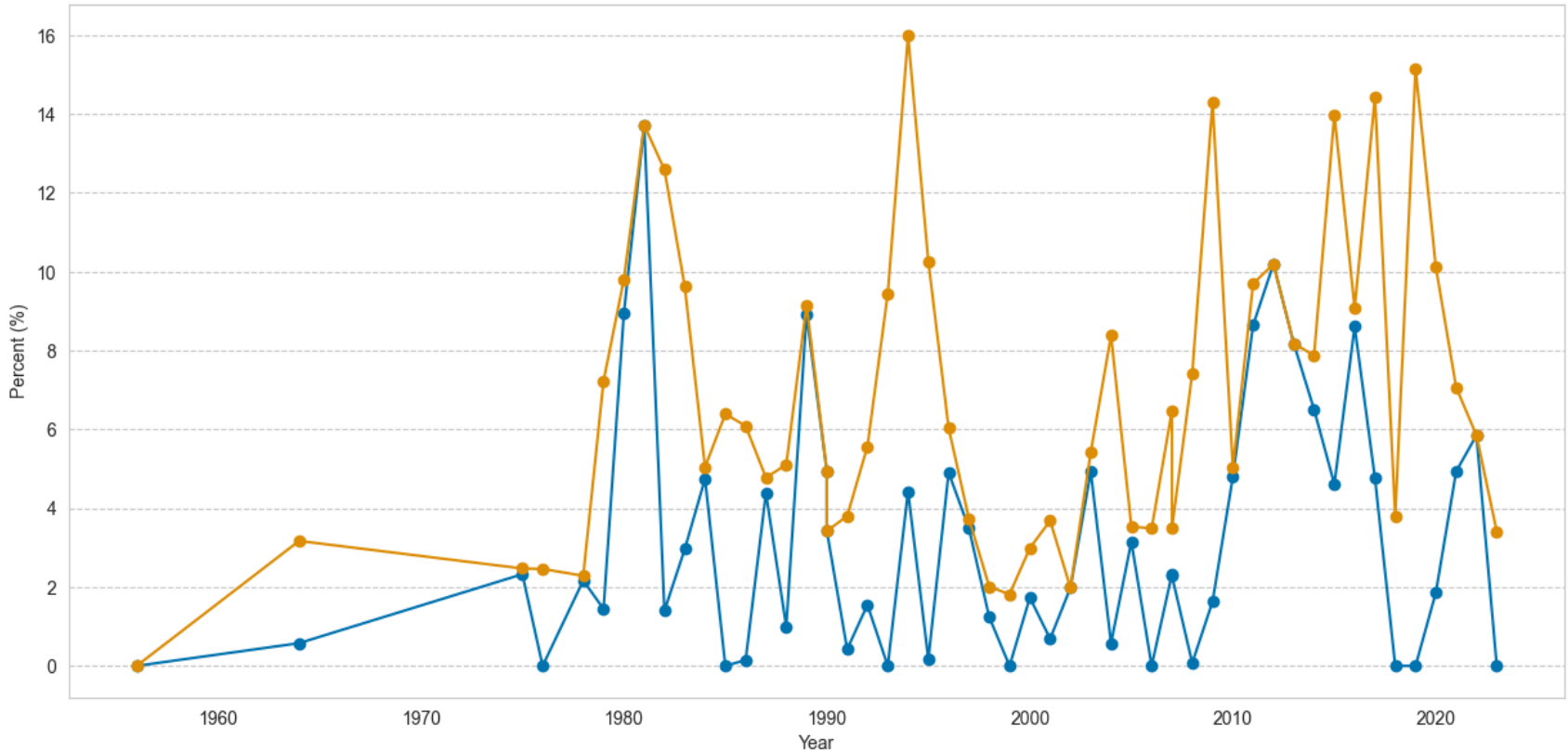
```
Monetary Policy_pct
Inflation_pct
Fiscal Policy_pct
Housing Market_pct
Debt and Deficits_pct
```



In [149…

```
# big plot
plt.figure(figsize=(15, 7))

plt.plot(time_series, dfn_year["fiscal_monetary_pct"], label="Fiscal and Monetary Policy", marker='o')

# include fiscal policy alone
plt.plot(time_series, dfn_year["Fiscal Policy_pct"], label="Fiscal Policy", marker='o')

# include monetary policy alone
# plt.plot(time_series, dfn_year["Monetary Policy_pct"], label="Monetary Policy")
```

```
plt.xlabel("Year")
plt.ylabel("Percent (%)")

# turn off the grid
plt.grid(False)
# only show y-axis grid
plt.grid(axis='y', linestyle='--')
```



Are they coordinating fiscal and monetary policies to smoothen the cycle?

In [114…

```
# get average percentage across all years by topic
tab = dfn_year[sorted_columns].mean().sort_values(ascending=False)

# remove _pct from the tab.index
tab.index = tab.index.str.replace("_pct", "")
```

```
# round to four digits
tab = tab.round(2)

with open(fr"{root}/estimation/topic-models/2024-10-25/average_percentage.tex", "w") as f:
    f.write('\\begin{tabular}{lc}\n')
    f.write('\\toprule\n')
    f.write('Topic & Average Percentage (\\%) \\\\\n')
    f.write('\\midrule\n')
    for i in range(len(tab)):
        f.write(f'{tab.index[i]} & {tab[i]} \\\\\n')
    f.write('\\bottomrule\n')
    f.write('\\end{tabular}\n')

tab
```

```
/var/folders/rf/jrsrkjxn4rl0m0bmrmpys7b40000gq/T/ipykernel_34660/2234148903.py:16: FutureWarning: Series.__
getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treate
d as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`
  f.write(f'{tab.index[i]} & {tab[i]} \\\\\n')
```

Out[114…

```
Monetary Policy        9.97
Fiscal Policy          6.77
Inflation              5.65
Debt and Deficits      3.36
Financial Stability    2.13
Housing Market         1.95
Stock Market           1.35
Demand Shocks          0.64
Supply Shocks          0.51
Geopolitics            0.43
Climate Change         0.19
dtype: float64
```

In [116…

```
dffm = dfn[(dfn["Fiscal Policy_yn"] == 1) & (dfn["Monetary Policy_yn"] == 1)]
```

In [ ]:

```
dffm["text"].index
```

Out[ ]:

```
Index([  822,   824,   827,  1121,  1122,  1417,  1553,  1699,  1747,  1884,
       ...
       54886, 54974, 55012, 55200, 55558, 55621, 55678, 55699, 55737, 55739],
      dtype='int64', length=601)
```

In [ ]:
```
output = pd.DataFrame()

# some judgement to avoid conflicts
# TBA

# a faster way: df_line = []
df_list = []

for i in dfn[(dfn["Fiscal Policy_yn"] == 1) & (dfn["Monetary Policy_yn"] == 1)].index:
    dialog = []
    temp = dfn.loc[i - 5 : i + 20][["speaker", "text"]]
    temp["i"] = i
    # with open(fr"{root}/estimation/topic-models/2024-10-25/dffm_dialogs/monetary_fiscal_lines.txt", 'w')
    #     for j, row in temp.iterrows():
    #         f.write(f"{j}\t{row['speaker']}\t{row['text']} \n")
    df_list.append(temp)

output = pd.concat(df_list)
output.to_csv(
    rf"{root}/estimation/topic-models/2024-10-25/dffm_dialogs/monetary_fiscal_lines.csv",
    index=False,
)
```

In [38]:
```
dfn.loc[822:830]
```

Out[38]:

	section	speaker	text	meta_data	check_typos	file	year	Fiscal Policy_ans	Fiscal Policy_yn
822	NaN	Senator CLARK.	'hank you, Mr. Chairman. I have no questions ...	NaN	No	hrg-1964-bcs-0011_from_1_to_46.csv	1964	Yes. The fiscal policies of this administratio...	1
823	NaN	Mr. ROBERTSON.	Very much so.	NaN	NaN	hrg-1964-bcs-0011_from_1_to_46.csv	1964	No.	0
824	NaN	Senator CLARK.	It ;as been the view of some of us- I am sure ...	NaN	No	hrg-1964-bcs-0011_from_1_to_46.csv	1964	Yes. "It would be desirable for the Federal Re...	1
825	NaN	Mr. ROBERTSON.	In response, Senator Clark, I would say that ...	NaN	NaN	hrg-1964-bcs-0011_from_1_to_46.csv	1964	No.	0
826	NaN	Senator CLARK.	And also the possibility, the threat of infla...	NaN	No	hrg-1964-bcs-0011_from_1_to_46.csv	1964	No.	0
827	NaN	Mr. ROBERTSON.	There is a threat, We have now started on the...	NaN	NaN	hrg-1964-bcs-0011_from_1_to_46.csv	1964	Yes. "We have now started on the policy of usi...	1
828	NaN	Senator CLARK.	In your own thinking with respect to the mone...	NaN	No	hrg-1964-bcs-0011_from_1_to_46.csv	1964	No.	0

	section	speaker	text	meta_data	check_typos	file	year	Fiscal Policy_ans	Fiscal Policy_yn
829	NaN	Mr. ROBERTSON.	I have been in Government 37 years and I have...	NaN	No	hrg-1964-bcs-0011_from_1_to_46.csv	1964	No.	0
830	NaN	Senator CLARK.	Let's make it 2 or 3 years.	NaN	NaN	hrg-1964-bcs-0011_from_1_to_46.csv	1964	No.	0

9 rows × 41 columns

In [ ]:
```
# sentiment:
# > 0.33 or < -0.33: number & one sentence supporting
# middle: number & one sentence supporting but feel free to leave it blank

# query 1:
# discrete numbers: -1, 0 and 1

# query 2:
# continuous numbers between -1 and 1 but with no example in system msg

# query 3:
# continuous numbers between -1 and 1 with example in system msg

# from small samples (10) first
```

In [ ]:
```
dfn.to_csv(rf"{root}/estimation/topic-models/2024-10-25/dfn.csv", index=True)
```

In [ ]:
```
dffm.to_csv(rf"{root}/estimation/topic-models/2024-10-25/dffm.csv", index=True)
```

In [206…
```
dffm.sample(10).text.values

# dffm.loc[dffm.text.str.contains('budgetary situation, in a sense, is as well synchronized with monetary p
# dfn.loc[5570:5590]

dffm.loc[dffm.text.str.contains("I have got to go preside and Mr. Byrd is in the chair, so I can't be late
dfn.loc[16490:16520, ['speaker', 'text']].values
```

```
# get the exchange with f'{speaker}: {text}' format and concatenate them
# print(dffm.loc[16490:16520, ['speaker', 'text']].apply(lambda x: f'{x["speaker"]}: {x["text"]}', axis=1)
dffm.loc[16490:16520]
```

Out[206…

	section	speaker	text	meta_data	check_typos	file	year	Fiscal Policy_ans	Fis Policy_
16493	OPENING STATEMENT OF SENATOR GRAMM	Senator GRAMM.	Thank you, Mr. Chairman. Chairman Greenspan, ...	NaN	No	hrg-1990-bhu-0054_from_1_to_105.csv	1990	Yes. "I'd just like to get you at some point, ...	
16495	NaN	Senator SANFORD.	I have got to go preside and Mr. Byrd is in t...	NaN	No	hrg-1990-bhu-0054_from_1_to_105.csv	1990	Yes. "I think the question is, if we continue ...	
16500	OPENING STATEMENT OF SENATOR BOND	Senator BOND.	Well, I'll tell you what. Once in a while you...	Laughter.	No	hrg-1990-bhu-0054_from_1_to_105.csv	1990	Yes. "I would suggest that your prepared state...	
16502	NaN	Senator SHELBY.	Mr. Chairman, I have a written statement that...	The complete prepared statement of Senator She...	No	hrg-1990-bhu-0054_from_1_to_105.csv	1990	Yes. "But what's bothering me is bothering a l...	
16514	NaN	Chairman GREENSPAN.	As usual, Mr. Chairman, I am pleased to be he...	NaN	No	hrg-1990-bhu-0054_from_1_to_105.csv	1990	Yes. "Major, substantive, credible cuts in the...	

5 rows × 41 columns

In [ ]:
```
# -----------------------
# Environment Setup & Packages
# -----------------------

import os
import re
import sys
import json
import io
import logging
import time
from glob import glob
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
from nltk.tokenize import word_tokenize
import tiktoken
import pyfixest as pf
from openai import OpenAI
import numpy as np
from joblib import Parallel, delayed
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
from collections import Counter, defaultdict
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
STOPWORDS= list(ENGLISH_STOP_WORDS)

stopwords1 = ["0o", "0s", "3a", "3b", "3d", "6b", "6o", "a", "a1", "a2", "a3", "a4", "ab", "able", "about"
stopwords2 = ['chairman','senator','thank','little','bit','just','chair']
STOPWORDS = STOPWORDS  +stopwords1+stopwords2


from nltk.stem.snowball import SnowballStemmer
stemmer = SnowballStemmer("english")  # Porter2
# Sparse matrix
from scipy.sparse import csr_matrix, save_npz

import numpy as np
import pandas as pd
```

```python
from scipy import sparse
from scipy.sparse import load_npz
import statsmodels.api as sm

from joblib import Parallel, delayed
from sklearn.preprocessing import OneHotEncoder
from statsmodels.stats.multitest import multipletests

from scipy.stats import norm


logging.getLogger("openai").setLevel(logging.ERROR)
logging.getLogger("httpx").setLevel(logging.ERROR)

root = '.'

# Initialize OpenAI API client
model = "gpt-4o-mini"
client = OpenAI(api_key = jingoo_api_key)

chariman_list =    ['william mcchesney martin jr',
    'arthur f burns',
    'g william miller',
    'paul a volcker',
    'alan greenspan',
    'ben s bernanke',
    'janet l yellen',
    'jerome h powell',
]


def cosine_similarity_matrix(list1, list2):
    """
    list1: List of lists, shape (A, B)
    list2: List of lists, shape (C, B)
    返回: A×C 的余弦相似度矩阵
    """
    X = np.array(list1, dtype=np.float32)  # A×B
    Y = np.array(list2, dtype=np.float32)  # C×B

    # 每行的L2范数
    nx = np.linalg.norm(X, axis=1, keepdims=True)  # A×1
```

```
    ny = np.linalg.norm(Y, axis=1, keepdims=True)  # C×1

    # # 避免除0
    # nx[nx == 0] = 1.0
    # ny[ny == 0] = 1.0

    # 矩阵乘法 + 广播除法
    sim = (X @ Y.T) / (nx @ ny.T)  # A×C
    return sim


def clean_and_tokenize(text: str):
    if not isinstance(text, str):
        return []
    s = text

    # Remove non-spoken parenthetical insertions like (Applause), [Laughter], etc.
    s = re.sub(r"\([^)]*\)", " ", s)
    s = re.sub(r"\[[^\]]*\]", " ", s)

    # Lowercase
    s = s.lower()

    # Delete hyphens and apostrophes (per paper)
    s = s.replace("-", "").replace("’", "").replace("'", "")

    # Replace all other punctuation with spaces
    s = re.sub(r"[^a-z0-9\s]", " ", s)

    # Collapse whitespace
    s = re.sub(r"\s+", " ", s).strip()

    # Tokenize
    tokens = s.split()
    # Remove stopwords
    tokens = [t for t in tokens if t not in STOPWORDS]

    tokens = [stemmer.stem(t) for t in tokens]

    return tokens

def bigrams_from_tokens(tokens):
```

```
    # Return list of consecutive two-word phrases (bigrams)
    if len(tokens) < 2:
        return []
    return [f"{tokens[i]} {tokens[i+1]}" for i in range(len(tokens)-1)]
```

In [2]:
```
data = pd.read_csv('./embedding_output/long_text_embedding_finished.csv')
data = data[~data['embedding'].isna()]
party_data = pd.read_excel('match_list_update.xlsx')
party_data['year'] = party_data['year'].astype(int)
name_list = pd.read_excel('named_list.xlsx')
data = pd.merge(data[['date', 'file', 'section', 'speaker', 'text', 'meta_data',
       'check_typos', 'year', 'word_cnt', 'Level',
       'note', 'is_nomination', 'embedding']],
       party_data[['speaker', 'year', 'file', 'full_name', 'Party ', 'ID']], on=['speaker','year','file'],
# data['embedding'] = data['embedding'].apply(lambda x:json.loads(x))
data['full_name'] = data['full_name'].str.lower()
```

In [3]:
```
# biggram building

data_politician = data[data['ID']=='politician'].copy()

# filter out the too short textual content
data_politician =data_politician [data_politician['word_cnt'] >= 30]

# Assign a stable doc_id
data_politician = data_politician.reset_index(drop=True)
data_politician["doc_id"] = np.arange(len(data_politician))
```

In [4]:
```
# s compute using embedding
data_politician['embedding'] = data_politician['embedding'].apply(lambda x:json.loads(x))
party_means =data_politician .groupby(['year','Party '])['embedding'].apply(lambda xs:np.mean(np.stack(xs,a
party_means = party_means[['Democratic','Republican']].dropna(subset=['Democratic','Republican'])
party_means['w'] = party_means['Democratic']-party_means['Republican']
party_means['w_norm'] = party_means['w'].apply(lambda v:np.linalg.norm(v) if v is not None else np.nan)
party_means = party_means.rename(columns= {'Democratic':'mu_d','Republican':'mu_r'})
data_party = pd.merge(data_politician,party_means,on = 'year',how = 'left')
def score_row(embedding, mu_D, w, w_norm):
    # s = ((x - mu_D) · w) / ||w||
    if w_norm is None or w_norm == 0 or embedding is None or mu_D is None or w is None:
        return np.nan
    return float(np.dot(embedding - mu_D, w) / w_norm)
```

```
for col in ['embedding','mu_d','w','w_norm']:
    data_party = data_party[~data_party[col].isna()]

data_party["s"] = [
    score_row(x, mD, wv, wn)
    for x, mD, wv, wn in zip(data_party["embedding"], data_party["mu_d"], data_party["w"], data_party["w_no
]


data_party[['date', 'file', 'section', 'speaker', 'meta_data',
       'check_typos', 'year', 'word_cnt', 'Level', 'note', 'is_nomination',
        'full_name', 'Party ', 'ID', 'doc_id', 'mu_d', 'mu_r', 'w',
       'w_norm', 's']].to_csv('./inter_result/s_compute_via_embedding.csv',index = False)
```

In [5]:
```
data_politician=data_politician[data_politician['doc_id'].isin(data_party['doc_id'])]
```

In [6]:
```
doc_bigrams = []         # list of list-of-bigrams
global_bigram_counts = Counter()
doc_freq = Counter()     # number of docs each bigram appears in

for _, row in data_politician.iterrows():
    tokens = clean_and_tokenize(row["text"])
    bigs = bigrams_from_tokens(tokens)
    doc_bigrams.append(bigs)
    # Update global counts
    global_bigram_counts.update(bigs)
    # Update document frequency
    for bg in set(bigs):
        doc_freq[bg] += 1

# ----------------------------
# 3) Frequency filters (adapted from the paper)
#    - min_total_count: total occurrences across corpus
#    - min_doc_freq   : number of unique (speaker×file) docs containing the bigram
#    - min_count_in_any_doc: appears at least this many times in at least one doc
# ----------------------------

# hype follow Partisan Corporate Speech, I thnk could do farthey modify
min_total_count =40
min_doc_freq =  20
min_count_in_any_doc = 2
```

```
# Precompute max count per doc for each bigram
bigram_max_in_doc = Counter()
for bigs in doc_bigrams:
    c = Counter(bigs)
    for bg, cnt in c.items():
        if cnt > bigram_max_in_doc[bg]:
            bigram_max_in_doc[bg] = cnt


# Build filtered vocabulary
vocab = []
for bg, tot in global_bigram_counts.items():
    if not re.search(r'\d',bg): # remove number
        if tot < min_total_count:
            continue
        if doc_freq[bg] < min_doc_freq:
            continue
        if bigram_max_in_doc[bg] < min_count_in_any_doc:
            continue

        vocab.append(bg)

vocab = sorted(vocab)
v_index = {bg:i for i, bg in enumerate(vocab)}
```

In [7]:

```
rows = []
cols = []
data_count = []

for d, bigs in enumerate(doc_bigrams):
    c = Counter(bigs)
    for bg, cnt in c.items():
        j = v_index.get(bg)
        if j is None:
            continue
        rows.append(d)
        cols.append(j)
        data_count.append(cnt)

if len(vocab) > 0:
    dtm = csr_matrix((data_count, (rows, cols)), shape=(len(data_politician), len(vocab)), dtype=np.int32)
```

```
else:
    # Empty matrix if filters are too strict for the demo
    dtm = csr_matrix((len(data_politician), 0), dtype=np.int32)
```

In [8]:
```
# sasve intermide result
os.makedirs('./inter_result/',exist_ok=True)
vocab_df = pd.DataFrame({
    "bigram": vocab,
    "total_count": [global_bigram_counts[bg] for bg in vocab],
    "doc_freq": [doc_freq[bg] for bg in vocab],
    "max_in_single_doc": [bigram_max_in_doc[bg] for bg in vocab],
}).sort_values(["total_count","doc_freq"], ascending=False)

vocab_df.reset_index(drop = True).to_csv('./inter_result/vocab.csv',index = True)


doc_index = data_politician[['doc_id','file','full_name','Party ']]
doc_index.to_csv('./inter_result/doc_index.csv',index = False)

dtm_path = "./inter_result/dtm_csr.npz"
save_npz(dtm_path, dtm)
```

In [9]:
```
DTM_PATH = "./inter_result/dtm_csr.npz"
META_CSV = "./inter_result/s_compute_via_embedding.csv"        # must include column 's' (your partisan sc
INDEX_CSV = './inter_result/doc_index.csv'
terms_table= pd.read_csv('./inter_result/vocab.csv')
terms = terms_table['bigram'].to_list()

MIN_DOCS_WITH_TERM = 10           # skip ultra-rare terms
N_JOBS = -1                       # parallelism; -1 = all cores
RIDGE_ALPHA = 0.0                 # 0.0 = unregularized GLM. (See note below about regularization.)
```

In [10]:
```
Y = load_npz(DTM_PATH)       # counts matrix
n_docs, n_terms = Y.shape

print(n_docs, n_terms)

df_s = pd.read_csv(META_CSV)
```

```
19432 881
```

In [11]:
```
N = np.asarray(Y.sum(axis=1)).ravel()
# Prevent log(0)
N_safe = np.where(N > 0, N, 1e-12)
logN = np.log(N_safe)
```

In [12]:
```
df_s['year'] = df_s['date'].apply(lambda x:x[:4])
```

In [13]:
```
feature_cols_exclude = {'s'}  # we'll add s separately
candidate_cols = ['year']
```

In [14]:
```
if candidate_cols:
    ohe = OneHotEncoder(drop='first', sparse_output=True, handle_unknown='ignore')
    X_cat = ohe.fit_transform(df_s[candidate_cols])
else:
    X_cat = sparse.csr_matrix((n_docs, 0), dtype=float)
```

In [15]:
```
s_vec = df_s['s'].to_numpy(dtype=float).reshape(-1, 1)
```

In [16]:
```
X_left = sparse.csr_matrix(s_vec)  # s
X = sparse.hstack([X_left, X_cat], format='csr')
X_cols = ['s'] + ['year']
```

In [17]:
```
j = 0
y = Y[:, j].toarray().ravel()  # counts for term j


# statsmodels needs dense arrays; we'll pass X as dense for simplicity.
# For large data, consider converting only once or using sparse-aware routines.
X_dense = X.toarray()

# Build GLM with offset = logN
# Note: GLM.fit_regularized exists but offset support can be finicky. We use unregularized by default.
model = sm.GLM(y, sm.add_constant(X_dense, has_constant='add'),
               family=sm.families.Poisson(),
               offset=logN)

# Robust (HC1) covariance to reduce sensitivity to mild misspec
try:
    res = model.fit(cov_type='HC1', maxiter=100)
```

```
except Exception as e:
    # Fallback to default if robust fails
    res = model.fit(maxiter=100)

# Coef order: [const, 's', rest...]
params = res.params
b_s = params[1] if len(params) > 1 else np.nan

# Robust SE if available
try:
    se = res.bse[1]
except Exception:
    se = np.nan

# z & p
if np.isfinite(b_s) and np.isfinite(se) and se > 0:
    z = b_s / se
    # two-sided p from normal approx
    p = 2 * (1 - norm.cdf(abs(z)))
else:
    z, p = np.nan, np.nan
```

In [18]:

```
# ----------------------------
# 3) Per-term Poisson GLM with offset
# ----------------------------
def fit_one_term(j):
    """
    Fit Poisson GLM for term j with log(N) offset.
    Return: (term, beta_s, se_s, z_s, p_s, df_resid, n_nz)
    """
    y = Y[:, j].toarray().ravel()  # counts for term j

    # Quick skip: if the term is extremely rare across docs, it won't be stable
    # You can count number of non-zero docs:
    nnz = int((y > 0).sum())
    if nnz < MIN_DOCS_WITH_TERM:
        return (terms[j], np.nan, np.nan, np.nan, np.nan, np.nan, nnz)

    # statsmodels needs dense arrays; we'll pass X as dense for simplicity.
    # For large data, consider converting only once or using sparse-aware routines.
    X_dense = X.toarray()
```

```
    # Build GLM with offset = logN
    # Note: GLM.fit_regularized exists but offset support can be finicky. We use unregularized by default.
    model = sm.GLM(y, sm.add_constant(X_dense, has_constant='add'),
                   family=sm.families.Poisson(),
                   offset=logN)

    # Robust (HC1) covariance to reduce sensitivity to mild misspec
    try:
        res = model.fit(cov_type='HC1', maxiter=100)
    except Exception as e:
        # Fallback to default if robust fails
        res = model.fit(maxiter=100)

    # Coef order: [const, 's', rest...]
    params = res.params
    b_s = params[1] if len(params) > 1 else np.nan

    # Robust SE if available
    try:
        se = res.bse[1]
    except Exception:
        se = np.nan

    # z & p
    if np.isfinite(b_s) and np.isfinite(se) and se > 0:
        z = b_s / se
        # two-sided p from normal approx
        p = 2 * (1 - norm.cdf(abs(z)))
    else:
        z, p = np.nan, np.nan

    return (terms[j], b_s, se, z, p, res.df_resid, nnz)

# Parallel over terms
results = Parallel(n_jobs=N_JOBS, verbose=10)(
    delayed(fit_one_term)(j) for j in range(n_terms)
)

res_df = pd.DataFrame(results, columns=[
    "term", "beta_s", "se_s", "z_s", "p_s", "df_resid", "docs_with_term"
])
```

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done   2 tasks      | elapsed:    9.1s
[Parallel(n_jobs=-1)]: Done   9 tasks      | elapsed:   15.8s
[Parallel(n_jobs=-1)]: Done  16 tasks      | elapsed:   21.6s
[Parallel(n_jobs=-1)]: Done  25 tasks      | elapsed:   30.3s
[Parallel(n_jobs=-1)]: Done  34 tasks      | elapsed:   38.6s
[Parallel(n_jobs=-1)]: Done  45 tasks      | elapsed:   47.6s
[Parallel(n_jobs=-1)]: Done  56 tasks      | elapsed:   59.7s
[Parallel(n_jobs=-1)]: Done  69 tasks      | elapsed:  1.2min
[Parallel(n_jobs=-1)]: Done  82 tasks      | elapsed:  1.4min
[Parallel(n_jobs=-1)]: Done  97 tasks      | elapsed:  1.6min
[Parallel(n_jobs=-1)]: Done 112 tasks      | elapsed:  1.9min
[Parallel(n_jobs=-1)]: Done 129 tasks      | elapsed:  2.1min
[Parallel(n_jobs=-1)]: Done 146 tasks      | elapsed:  2.4min
[Parallel(n_jobs=-1)]: Done 165 tasks      | elapsed:  2.7min
[Parallel(n_jobs=-1)]: Done 184 tasks      | elapsed:  3.0min
[Parallel(n_jobs=-1)]: Done 205 tasks      | elapsed:  3.3min
[Parallel(n_jobs=-1)]: Done 226 tasks      | elapsed:  3.6min
[Parallel(n_jobs=-1)]: Done 249 tasks      | elapsed:  4.0min
[Parallel(n_jobs=-1)]: Done 272 tasks      | elapsed:  4.3min
[Parallel(n_jobs=-1)]: Done 297 tasks      | elapsed:  4.7min
[Parallel(n_jobs=-1)]: Done 322 tasks      | elapsed:  5.1min
[Parallel(n_jobs=-1)]: Done 349 tasks      | elapsed:  5.6min
[Parallel(n_jobs=-1)]: Done 376 tasks      | elapsed:  6.0min
[Parallel(n_jobs=-1)]: Done 405 tasks      | elapsed:  6.5min
[Parallel(n_jobs=-1)]: Done 434 tasks      | elapsed:  6.9min
[Parallel(n_jobs=-1)]: Done 465 tasks      | elapsed:  7.4min
[Parallel(n_jobs=-1)]: Done 496 tasks      | elapsed:  7.9min
[Parallel(n_jobs=-1)]: Done 529 tasks      | elapsed:  8.4min
[Parallel(n_jobs=-1)]: Done 562 tasks      | elapsed:  8.9min
[Parallel(n_jobs=-1)]: Done 597 tasks      | elapsed:  9.4min
[Parallel(n_jobs=-1)]: Done 632 tasks      | elapsed: 10.0min
[Parallel(n_jobs=-1)]: Done 669 tasks      | elapsed: 10.6min
[Parallel(n_jobs=-1)]: Done 706 tasks      | elapsed: 11.2min
[Parallel(n_jobs=-1)]: Done 745 tasks      | elapsed: 12.0min
[Parallel(n_jobs=-1)]: Done 784 tasks      | elapsed: 12.6min
[Parallel(n_jobs=-1)]: Done 825 tasks      | elapsed: 13.4min
[Parallel(n_jobs=-1)]: Done 866 tasks      | elapsed: 14.2min
[Parallel(n_jobs=-1)]: Done 881 out of 881 | elapsed: 14.4min finished
```

In [19]:
```
# beta s hihght means more democtirc
```

```
res_df
```

Out[19]:

	term	beta_s	se_s	z_s	p_s	df_resid	docs_with_term
0	feder reserv	1.657399	0.941740	1.759933	0.078419	19370	41
1	monetari polici	-1.339495	1.214869	-1.102584	0.270208	19370	38
2	reserv board	2.512434	0.814072	3.086256	0.002027	19370	49
3	unit state	1.928406	1.066441	1.808263	0.070566	19370	40
4	econom growth	-0.881825	1.008501	-0.874391	0.381905	19370	55
...	...	...	...	...	...	...	...
876	audit fed	0.417727	0.670520	0.622991	0.533291	19370	157
877	feder hous	-5.528089	1.169380	-4.727369	0.000002	19370	39
878	kansa citi	-2.484074	1.190059	-2.087353	0.036856	19370	60
879	execut order	0.776804	0.725585	1.070590	0.284354	19370	130
880	financi literaci	5.000009	1.142049	4.378103	0.000012	19370	43

881 rows × 7 columns

In [20]:

```
# ----------------------------
# 4) Multiple testing correction (BH-FDR)
# ----------------------------
mask = res_df["p_s"].notna()
if mask.any():
    rej, p_adj, _, _ = multipletests(res_df.loc[mask, "p_s"], method='fdr_bh')
    res_df.loc[mask, "p_adj_bh"] = p_adj
    res_df.loc[mask, "significant_bh"] = rej
else:
    res_df["p_adj_bh"] = np.nan
    res_df["significant_bh"] = False

# # Rank by |beta|, then by p-value
# res_df["abs_beta"] = res_df["beta_s"].abs()
# res_df = res_df.sort_values(["abs_beta", "p_s"], ascending=[False, True])
```

```
# # Save
# res_df.to_csv("mnir_term_loadings.csv", index=False)
# print("Saved: mnir_term_loadings.csv")
# print(res_df.head(20))
```

In [21]:
```
res_df.to_csv('biagram_result_trail_version.csv',index = False)
```

In [22]:
```
res_df[res_df['significant_bh']].sort_values('beta_s')
```

Out[22]:

	term	beta_s	se_s	z_s	p_s	df_resid	docs_with_term	p_adj_bh	significant_bh
820	critic fed	-14.760238	4.598385	-3.209874	1.327931e-03	19370	44	6.428064e-03	True
845	price rise	-9.890687	2.216863	-4.461569	8.136163e-06	19370	32	8.849333e-05	True
250	consum spend	-9.778692	1.379249	-7.089868	1.342482e-12	19370	64	7.392040e-11	True
690	polici rate	-9.572494	1.136363	-8.423800	0.000000e+00	19370	20	0.000000e+00	True
11	wall street	-9.287882	1.118014	-8.307482	0.000000e+00	19370	139	0.000000e+00	True
...	...	...	...	...	...	...	...	...	...
689	extend period	6.680165	0.970715	6.881693	5.914602e-12	19370	73	2.605382e-10	True
388	semiannu monetari	7.240968	0.831342	8.709978	0.000000e+00	19370	60	0.000000e+00	True
841	financi regulatori	8.144978	2.162628	3.766241	1.657239e-04	19370	26	1.177441e-03	True
31	price stabil	9.489164	0.935589	10.142454	0.000000e+00	19370	62	0.000000e+00	True
10	unemploy rate	10.336085	1.075698	9.608725	0.000000e+00	19370	70	0.000000e+00	True

262 rows × 9 columns

In [ ]:
```
# sessionts by seensstion
# congression by congression
# between the elective
# two year by two year

#congressianlly dummy , baseline(?)
```